

27. Konceptuální návrh relačních databází, základní konstrukty, ER diagram, kardinalita, parcialita, závislost

Klíčové pojmy

- entita
- atribut
- vztah
- kardinalita
- parcialita
- závislost (slabé entity)
- primární klíč
- cizí klíč

A. Konceptuální návrh relačních databází

První fáze návrhu DB. Cíl: zachytit informační požadavky uživatelů, vytvořit model dat nezávislý na konkrétním DBMS. Hlavní nástroj: **Entity-Relationship model (ER model)**, graficky **ER diagram (ERD)**. Poskytuje abstraktní pohled na data a jejich vztahy.

B. Základní konstrukty ER modelu

1. **Entita:** Reálný objekt, osoba, místo nebo koncept, o kterém chceme ukládat data (např. `Student` , `Kniha`). V ERD: obdélník.
2. **Atribut:** Vlastnost nebo charakteristika entity (např. `Jméno` studenta, `E-mail`). V ERD: elipsa.
 - **Klíčový atribut (Primární klíč):** Atribut/skupina atributů, který jednoznačně identifikuje každou instanci entity (např. `ID_studenta`). V ERD: podtržený.
3. **Vztah:** Logická vazba nebo asociace mezi dvěma nebo více entitami (např. `Student` *studuje* `Předmět`). V ERD: kosočtverec.

C. Kardinalita, Parcialita a Závislost

Vztahy mezi entitami se řídí strukturálními omezeními:

1. **Kardinalita (Strukturální poměr vztahu):** Určuje max. počet instancí jednoho entitního typu, které se mohou účastnit vztahu s instancí druhého entitního typu.
 - **1:1 (Jeden k jednomu):** Jedna instance A je svázána s max. jednou B a naopak (např. `Občan` má `Cestovní pas`).
 - **1:N (Jeden k mnoha):** Jedna instance A s mnoha B, ale B s max. jednou A (např. `Fakulta` má `Studentů`).
 - **M:N (Mnoha k mnoha):** Jedna instance A s mnoha B a naopak (např. `Student` navštěvuje `Předmětů`).
2. **Parcialita (Členství / Účast ve vztahu):** Určuje, zda je účast entity ve vztahu povinná, nebo volitelná.
 - **Totální (Povinná) účast:** Každá instance entity se *musí* účastnit vztahu (např. každá `Objednávka` musí mít `Zákazníka`).
 - **Parciální (Volitelná) účast:** Instance entity se vztahu účastnit *může, ale nemusí* (např. `Zákazník` nemusí mít `Objednávku`).
3. **Závislost (Slabé entitní typy):** Popisuje situaci, kdy entita nemůže existovat sama o sobě.
 - **Silná entita:** Existuje nezávisle, má vlastní primární klíč.
 - **Slabá entita (Identifikačně/Existenčně závislá):** Nemůže existovat bez své nadřazené (silné) entity. Její identifikátor je složen z klíče nadřazené entity a vlastního parciálního klíče (např. `Místnost` je slabá entita závislá na `Budova`).

Praktické použití

- Návrh informačních systémů
- Komunikace se zákazníkem
- Dokumentace databáze
- Prevence redundance

Nejčastější chyby u zkoušky

- Příliš brzké řešení SQL bez konceptuálního návrhu.
- Nejasné kardinality.
- Entita zaměněná za atribut.

Typické zkouškové otázky

1. Co je entita, atribut a vztah?

- **Entita:** Reálný objekt/koncept, o kterém ukládáme data (např. `Student`). ERD: obdélník.
- **Atribut:** Vlastnost entity (např. `Jméno`). ERD: elipsa.
- **Vztah:** Logická vazba mezi entitami (např. `Student` *studuje* `Předmět`). ERD: kosočtverec.

2. Jak se převádí M:N vztah do tabulek?

- M:N vztah se převádí na **spojovací (asociační) tabulku**. Tato tabulka obsahuje jako primární klíč složený klíč z primárních klíčů obou původních entit, které se stávají cizími klíči. Může mít i vlastní atributy.

3. Co je primární a cizí klíč?

- **Primární klíč (PK):** Atribut/skupina atributů, jednoznačně identifikuje každý záznam v tabulce. Musí být jedinečný a `NOT NULL`. Zajišťuje entitní integritu.
- **Cizí klíč (FK):** Atribut/skupina atributů v jedné tabulce odkazující na primární klíč v jiné tabulce. Propojuje tabulky a zajišťuje referenční integritu (hodnota FK musí existovat jako PK v odkazované tabulce, nebo být `NULL`, je-li povoleno).

Shrnutí kapitoly

ER model je první krok dobrého návrhu databáze. Pomáhá pochopit doménu před tvorbou tabulek.

28. Normalizace, normální formy, funkční závislosti, aktualizací anomálie

Klíčové pojmy

- **Redundance:** Opakování dat.
- **Anomálie:** Nežádoucí efekty při DML operacích (vkládání, mazání, úpravy).
- **Funkční závislost** ($X \rightarrow Y$): Hodnota X jednoznačně určuje Y .
- **Determinant:** Atribut(y) na levé straně funkční závislosti (X).
- **Kandidátní klíč:** Minimální sada atributů jednoznačně identifikující řádek.
- **Primární klíč:** Vybraný kandidátní klíč.
- **1NF:** Atomické atributy, PK, žádné duplicitní řádky.
- **2NF:** 1NF + neklíčové atributy plně závislé na celém PK.
- **3NF:** 2NF + žádné tranzitivní závislosti neklíčových atributů na PK.
- **BCNF:** 3NF + každý determinant je kandidátní klíč.

A. Aktualizační anomálie

Problémy v špatně navržené tabulce:

- **Vkládání:** Nelze vložit data o entitě bez dat o jiné, na které je závislá.
- **Mazání:** Smazání dat o jedné entitě nechtěně smaže data o jiné.
- **Úprava:** Nutnost aktualizovat stejná data na více místech, riziko nekonzistence.

B. Funkční závislosti

Y je funkčně závislé na X ($X \rightarrow Y$), pokud pro každou hodnotu X existuje právě jedna hodnota Y .

- **Plná funkční závislost:** Y závisí na celém složeném klíči X , ne jen na jeho části.
- **Tranzitivní funkční závislost:** Pokud $X \rightarrow Y$ a $Y \rightarrow Z$, pak Z závisí na X tranzitivně.

C. Normální formy (1NF, 2NF, 3NF, BCNF)

Normalizace: Proces převodu datové struktury do vyšších NF pro odstranění anomálií a duplicit.

1. První normální forma (1NF)

- Všechny atributy atomické (nedělitelné).
- Definovaný primární klíč, žádné duplicitní řádky.
- *Porušení*: Více hodnot v jednom sloupci (např. "tel1, tel2").

2. Druhá normální forma (2NF)

- Splňuje 1NF.
- Všechny neklíčové atributy jsou plně funkčně závislé na celém primárním klíči (relevantní pro složený PK).
- *Porušení*: Neklíčový atribut závisí jen na části složeného PK.

3. Třetí normální forma (3NF)

- Splňuje 2NF.
- Žádný neklíčový atribut není tranzitivně závislý na primárním klíči (žádný neklíčový na jiném neklíčovém).
- *Porušení*: $A \rightarrow B \rightarrow C$, kde B, C jsou neklíčové atributy.

4. Boyce-Coddova normální forma (BCNF)

- Splňuje 3NF.
- Každý determinant je kandidátním klíčem.
- Přísnější než 3NF, řeší specifické případy s překrývajícími se kandidátními klíči.
- *Porušení*: Determinant funkční závislosti není kandidátní klíč.

D. Výhody a nevýhody normalizace

Výhody

- Odstranění redundance.

- Zajištění konzistence dat.
- Zlepšení datové kvality.
- Snazší údržba a jasná struktura.
- Dlouhodobá udržitelnost IS.

Nevýhody (Denormalizace)

- Snížení výkonu při čtení (více JOINů).
- Zvýšená složitost dotazů.
- Denormalizace se využívá v OLAP (datové sklady) pro rychlost čtení.

Nejčastější chyby u zkoušky

- Mechanické dělení tabulek bez pochopení závislostí.
- Neznalost kandidátních klíčů.
- Zaměňování normalizace a indexace.
- Ignorování BCNF.

Typické zkouškové otázky

1. Co je funkční závislost?

$X \rightarrow Y$: Hodnota atributu X jednoznačně určuje hodnotu atributu Y . X je determinant.

2. Vysvětlete 1NF, 2NF a 3NF.

- **1NF**: Atomické atributy, definovaný PK, žádné duplicitní řádky.
- **2NF**: 1NF + neklíčové atributy plně funkčně závislé na celém PK (pro složený PK).
- **3NF**: 2NF + žádné tranzitivní závislosti neklíčových atributů na PK.

3. Jaké anomálie normalizace odstraňuje?

Aktualizační anomálie: Vkládání (nelze vložit bez závislých dat), Mazání (nechtěné smazání závislých dat), Úprava (nutnost více aktualizací, riziko nekonzistence).

4. Co je BCNF a jaký je její hlavní rozdíl oproti 3NF?

BCNF je 3NF + každý determinant je kandidátní klíč. Je přísnější než 3NF a řeší případy, kdy 3NF nestačí (např. u tabulek s více překrývajícími se kandidátními klíči).

Shrnutí kapitoly

Normalizace v relačních DB pomocí funkčních závislostí odstraňuje redundanci a aktualizací anomálie (vkládání, mazání, úpravy) pro datovou konzistenci a integritu. Může snížit výkon při čtení (více JOINů), ale zlepšuje datovou kvalitu a udržitelnost. V OLAP systémech se pro rychlost dotazů využívá denormalizace.

29. Relační model, základní konstrukty, realizace vztahů, integritní omezení

Relační datový model (E. F. Codd, 1970) je základem většiny moderních databázových systémů. Je založen na matematickém pojmu relace, v praxi reprezentované jako tabulka.

A. Základní konstrukty relačního modelu

- **Relace (Tabulka):** Dvourozměrná datová struktura s pojmenovanými sloupci (atributy) a neuspořádanou množinou řádků (n-tic). Každá relace má unikátní název a neobsahuje duplicitní řádky.
- **Atribut (Sloupec):** Pojmenovaná vlastnost uchovávající data. Každý atribut má definovanou **doménu** (množinu povolených hodnot, datový typ). Hodnoty atributů by měly být atomické.
- **N-tice (Řádek, Záznam):** Konkrétní instance dat v tabulce, představující jeden záznam. Každá n-tice je unikátní. Pořadí řádků je teoreticky nedůležité.
- **Doména:** Množina všech povolených hodnot pro daný atribut (datový typ, rozsah, formát).
- **Schéma relace:** Logická definice struktury relace (název, seznam atributů s doménami). Příklad: `Student (ID: INT, Jméno: VARCHAR(100))`.
- **Instance relace:** Aktuální obsah relace (množina n-tic) v daném čase.
- **Stupeň relace (Arity):** Počet atributů (sloupců) v tabulce.
- **Kardinalita relace:** Počet n-tic (řádků) v tabulce.

B. Realizace vztahů v relačním modelu

Vztahy se realizují pomocí provázání hodnot klíčů v tabulkách.

1. Primární klíč (PK - Primary Key):

- Atribut nebo minimální množina atributů, která jednoznačně identifikuje každý řádek v tabulce.
- **Vlastnosti:** Unikátnost, `NOT NULL`, stabilita.

- **Kandidátní klíč:** Jakákoli minimální množina atributů, která může jednoznačně identifikovat řádek.
- **Složený PK:** PK tvořený dvěma nebo více atributy.

2. Cizí klíč (FK - Foreign Key):

- Atribut (nebo skupina atributů) v jedné tabulce (odkazující), který odkazuje na primární klíč (nebo kandidátní klíč) v tabulce jiné (odkazovaná/rodičovská).
- Propojuje tabulky a zajišťuje referenční integritu.

3. Přepis vztahů z ER diagramu do tabulek:

- **Vztah 1:N (Jeden k mnoha):** PK z tabulky na straně "1" (rodičovská) se vloží jako FK do tabulky na straně "N" (potomkovská).
 - Příklad: Fakulta (id_fakulty PK, nazev) , Student (id_studenta PK, jmeno, id_fakulty_fk FK) .
- **Vztah M:N (Mnoha k mnoha):** Vytvoří se **vazební (asociativní) tabulka**. Obsahuje minimálně dva FK, které odkazují na PK obou původních tabulek. Tyto FK obvykle tvoří **složený PK** vazební tabulky.
 - Příklad: Student (id_studenta PK, jmeno) , Predmet (id_predmetu PK, nazev) , Zapis_Predmetu (id_studenta_fk FK, id_predmetu_fk FK, znamka) . Složený PK: (id_studenta_fk, id_predmetu_fk) .
- **Vztah 1:1 (Jeden k jednomu):** PK jedné tabulky se vloží jako FK do druhé tabulky. Na tento FK se navíc uvalí podmínka **unikátnosti** (**UNIQUE**).
 - Příklad: Osoba (id_osoby PK, jmeno) , DetailOsoby (id_osoby_fk PK, UNIQUE, adresa) .

C. Integritní omezení

Pravidla, která databáze vynucuje pro zajištění přesnosti a konzistence dat.

1. Entitní integrita:

- **Pravidlo:** Každá tabulka musí mít PK. Hodnota PK musí být unikátní a **NOT NULL** .
- **Význam:** Zajišťuje jednoznačnou identifikaci každého záznamu.

2. Referenční integrita:

- **Pravidlo:** Hodnota FK v odkazující tabulce musí buď odpovídat existující hodnotě PK v odkazované tabulce, nebo musí být `NULL` (pokud je povoleno).
- **Význam:** Zabraňuje vzniku "osiřelých" záznamů.
- **Referenční akce (ON DELETE/ON UPDATE):**
 - `RESTRICT / NO ACTION` : Zakáže smazání/aktualizaci rodičovského záznamu, pokud na něj existují odkazy.
 - `CASCADE` : Smazání/aktualizace rodiče se kaskádově šíří na potomky.
 - `SET NULL` : Smazání/aktualizace rodiče nastaví FK potomků na `NULL` (pokud není `NOT NULL`).
 - `SET DEFAULT` : Smazání/aktualizace rodiče nastaví FK potomků na výchozí hodnotu.

3. Doménová integrita:

- **Pravidlo:** Hodnoty ve sloupci musí odpovídat definovanému datovému typu, rozsahu nebo seznamu hodnot.
- **Nástroje:** Datové typy, `NOT NULL` , `CHECK` podmínky, `DEFAULT` hodnota.

4. Uživatelská (Business) integrita:

- **Pravidlo:** Specifická pravidla definovaná na základě obchodní logiky, která nelze vyjádřit základními integritními omezeními.
- **Realizace:** `UNIQUE` constraint, Spouště (Triggery), Uložené procedury.
- **Příklad:** "Student si může zapsat maximálně 5 předmětů za semestr."

Typické zkouškové otázky (zkrácené odpovědi)

1. Popište základní konstrukty relačního modelu a vysvětlete jejich význam.

- **Relace (Tabulka):** Dvourozměrná struktura dat (sloupce, řádky).
- **Atribut (Sloupec):** Pojmenovaná vlastnost, má doménu.
- **N-tice (Řádek):** Konkrétní záznam.

- **Doména:** Množina povolených hodnot pro atribut.
- **Schéma relace:** Struktura relace (název, atributy, domény).
- **Instance relace:** Aktuální obsah relace.

2. Jak se realizují vztahy 1:N a M:N v relačním modelu? Uveďte příklady.

- **1:N:** PK z "1" tabulky jako FK do "N" tabulky.
 - Schéma: `Fakulta (id_fakulty PK)` , `Student (id_studenta PK, id_fakulty_fk FK)` .
- **M:N:** Vazební tabulka s FK na PK obou tabulek, tvořící složený PK.
 - Schéma: `Student (id_studenta PK)` , `Predmet (id_predmetu PK)` , `Zapis_Predmetu (id_studenta_fk FK, id_predmetu_fk FK)` .

3. Vyjmenujte a vysvětlete základní typy integritních omezení v relačním modelu.

- **Entitní integrita:** PK unikátní a `NOT NULL` .
- **Referenční integrita:** FK odkazuje na existující PK nebo je `NULL` .
- **Doménová integrita:** Hodnoty odpovídají datovému typu/rozsahu.
- **Uživatelská (Business) integrita:** Specifická obchodní pravidla (např. `UNIQUE` , trigger).

4. Vysvětlete referenční akce `CASCADE` , `SET NULL` a `RESTRICT` při definici cizího klíče.

- `CASCADE` : Smazání/aktualizace rodiče se šíří na potomky.
- `SET NULL` : Smazání/aktualizace rodiče nastaví FK potomků na `NULL` .
- `RESTRICT / NO ACTION` : Zakáže smazání/aktualizaci rodiče, pokud existují potomci.

30. CRUD operace, SQL DDL, SQL DML, SQL dotazy, Indexy, Pohledy, Uložené procedury a funkce, Triggery. Transakce, ACID vlastnosti, úrovně izolace transakcí.

CRUD Operace

Základní operace pro interakci s daty v perzistentních datových strukturách:

- **Create:** Vytvoření nového záznamu. SQL: `INSERT`
- **Read:** Čtení existujících záznamů. SQL: `SELECT`
- **Update:** Modifikace existujících záznamů. SQL: `UPDATE`
- **Delete:** Smazání existujících záznamů. SQL: `DELETE`

Rozdělení jazyka SQL

- **SQL DDL (Data Definition Language):** Definuje a spravuje strukturu databázových objektů (schéma).
 - `CREATE` : Vytvoří nový objekt (tabulka, index, pohled). Příklad: `CREATE TABLE student (id INT PRIMARY KEY);`
 - `ALTER` : Změní strukturu existujícího objektu. Příklad: `ALTER TABLE student ADD COLUMN email VARCHAR(255);`
 - `DROP` : Kompletně smaže objekt i s daty. Příklad: `DROP TABLE student;`
 - `TRUNCATE` : Rychle odstraní všechny řádky z tabulky, zachová strukturu.
- **SQL DML (Data Manipulation Language):** Manipuluje s daty uvnitř již definovaných struktur.
 - `SELECT` : Dotazování a čtení dat. Příklad: `SELECT jmeno FROM student WHERE vek > 20;`
 - `INSERT` : Vkládání nových řádků. Příklad: `INSERT INTO student (id, jmeno) VALUES (1, 'Jan');`

- `UPDATE` : Modifikace existujících řádků. Příklad: `UPDATE student SET vek = 23 WHERE id = 1;`
- `DELETE` : Mazání konkrétních řádků. Příklad: `DELETE FROM student WHERE vek < 20;`

Základní operace relační algebry v SQL

- **Selekce (Restrikce):** Výběr řádků, které splňují podmínku. SQL: Klauzule `WHERE` .
 - Příklad: `SELECT FROM student WHERE mesto = 'Praha';`
- **Projekce:** Výběr konkrétních sloupců. SQL: Výpis sloupců za `SELECT` .
 - Příklad: `SELECT jmeno, email FROM student;`

Typy spojení tabulek (JOINS)

Propojení dat z více tabulek přes relace (PK/FK).

- `INNER JOIN` : Vrátí pouze řádky se shodou v obou tabulkách.
- `LEFT JOIN` (`LEFT OUTER JOIN`): Všechny řádky z levé tabulky, shodné z pravé. Bez shody `NULL` vpravo.
- `RIGHT JOIN` (`RIGHT OUTER JOIN`): Všechny řádky z pravé tabulky, shodné z levé. Bez shody `NULL` vlevo.
- `FULL JOIN` (`FULL OUTER JOIN`): Všechny řádky z obou tabulek. Bez shody `NULL` na chybějící straně.
- `CROSS JOIN` : Kartézský součin (každý řádek z první s každým z druhé).

Agregační funkce a seskupování

Agregační funkce zpracovávají skupinu řádků a vrací jednu souhrnnou hodnotu.

- `COUNT()` : Počet řádků/ne-NULL hodnot.
- `SUM()` : Součet hodnot.
- `AVG()` : Aritmetický průměr.
- `MIN()` : Minimální hodnota.
- `MAX()` : Maximální hodnota.
- `GROUP BY` : Seskupuje řádky s identickými hodnotami pro aplikaci agregačních funkcí.
- Příklad: `SELECT ID_Fakulty, COUNT(ID_Studenta) FROM Studenti GROUP BY`

ID_Fakulty;

- **HAVING** : Filtruje skupiny vytvořené **GROUP BY** na základě podmínek na agregační funkce.

- Příklad: `... HAVING COUNT(ID_Studenta) > 500;`

Množinové operace

Kombinují výsledky dvou **SELECT** dotazů (musí mít stejný počet a kompatibilní typy sloupců).

- **UNION** : Sjednocení, odstraní duplicity.
- **UNION ALL** : Sjednocení, zachová duplicity.
- **INTERSECT** : Průnik (řádky, které jsou v obou dotazech).
- **EXCEPT** (**MINUS**): Rozdíl (řádky v prvním, ale ne ve druhém dotazu).

Vnořené dotazy (Subqueries)

SELECT příkaz umístěný uvnitř jiného SQL příkazu.

- **Nekorelovaný**: Vnitřní dotaz je nezávislý, provede se jednou.

- Příklad: `SELECT jmeno FROM student WHERE prumer > (SELECT AVG(prumer) FROM student);`

- **Korelovaný**: Vnitřní dotaz se odkazuje na vnější, provede se pro každý řádek vnějšího dotazu.

Indexy, Pohledy, Uložené procedury a funkce, Triggery

- **Indexy**: Speciální vyhledávací struktury pro zrychlení **SELECT** dotazů (zejména s **WHERE** , **JOIN** , **ORDER BY**). Zpomalují **INSERT/UPDATE/DELETE** .
 - **Clustered Index**: Určuje fyzické pořadí dat, tabulka má jen jeden (často na PK).
 - **Non-clustered Index**: Neovlivňuje fyzické pořadí, obsahuje ukazatele na data, tabulka může mít více.
 - `CREATE INDEX idx_jmeno ON student (jmeno);`
- **Pohledy (Views)**: Virtuální tabulky založené na výsledku SQL dotazu. Neobsahují data.
 - Význam: Zjednodušení složitých dotazů, bezpečnost (omezení přístupu), konzistence.

- `CREATE VIEW studenti_praha AS SELECT jmeno FROM student WHERE mesto = 'Praha';`

- **Uložené procedury a funkce:** Předkompilované bloky SQL příkazů uložené v databázi.

- **Procedura:** Může přijímat parametry, nemusí vracet hodnotu. Pro provádění akcí.
- **Funkce:** Musí vrátit skalární hodnotu (nebo tabulku). Lze použít v SQL dotazech.
- Význam: Zvýšení výkonu, modularita, bezpečnost, snížení síťového provozu.
- `CREATE PROCEDURE/FUNCTION ...`

- **Triggery (Spouštěče):** Speciální uložené procedury, které se automaticky spustí v reakci na událost (`INSERT` , `UPDATE` , `DELETE`) na tabulce.

- Význam: Vynucování komplexních obchodních pravidel, auditování změn, udržování integrity.
- `CREATE TRIGGER po_vlozeni_studenta AFTER INSERT ON student FOR EACH ROW ...`

Transakce, ACID vlastnosti, úrovně izolace transakcí

- **Transakce:** Logická jednotka práce (jedna nebo více SQL operací). Buď se všechny operace provedou (`COMMIT`), nebo se žádná (`ROLLBACK`).

- Příkazy: `BEGIN TRANSACTION;` , `COMMIT;` , `ROLLBACK;` , `SAVEPOINT;`

- **ACID vlastnosti:** Zaručují spolehlivost transakcí.

- **Atomicity:** Transakce je nedělitelná; vše nebo nic.
- **Consistency:** Převeď databázi z jednoho konzistentního stavu do jiného.
- **Isolation:** Souběžné transakce se navzájem neovlivňují.
- **Durability:** Potvrzené změny jsou trvalé a přežijí selhání systému.

- **Problémy souběžnosti:**

- **Dirty Read:** Čtení nepotvrzených změn jiné transakce.
- **Non-repeatable Read:** Opakované čtení stejného řádku dává různé hodnoty.
- **Phantom Read:** Opakovaný dotaz vrací jiný počet řádků.

- **Úrovně izolace (od nejnižší po nejvyšší):**

1. `READ UNCOMMITTED` : Povoluje Dirty Read, Non-repeatable Read, Phantom Read.
2. `READ COMMITTED` : Zabraňuje Dirty Read. Povoluje Non-repeatable Read, Phantom Read.
3. `REPEATABLE READ` : Zabraňuje Dirty Read, Non-repeatable Read. Povoluje Phantom Read.
4. `SERIALIZABLE` : Zabraňuje všem problémům. Nejvyšší souběžnost.
5. Nastavení: `SET TRANSACTION ISOLATION LEVEL READ COMMITTED;`

Typické zkouškové otázky (Zkrácené odpovědi)

1. **CRUD operace a SQL mapování:** Create (`INSERT`), Read (`SELECT`), Update (`UPDATE`), Delete (`DELETE`). Základní interakce s daty.
2. **SQL DDL vs. SQL DML:**
 - DDL: Definuje strukturu (`CREATE TABLE` , `ALTER TABLE` , `DROP TABLE`).
 - DML: Manipuluje s daty (`SELECT` , `INSERT` , `UPDATE` , `DELETE`).
3. **Selekce a Projekce:**
 - Selekce: Výběr řádků dle podmínky (`WHERE`).
 - Projekce: Výběr sloupců (`SELECT sloupec`).
4. **Typy JOIN operací:**
 - `INNER JOIN` : Jen shody v obou.
 - `LEFT JOIN` : Vše zleva, shody zprava (jinak `NULL`).
 - `RIGHT JOIN` : Vše zprava, shody zleva (jinak `NULL`).
 - `FULL JOIN` : Vše z obou (jinak `NULL`).
5. **Agregační funkce, GROUP BY, HAVING:**
 - Agregace funkce (`COUNT` , `SUM`): Zpracují skupinu řádků na jednu hodnotu.
 - `GROUP BY` : Seskupuje řádky pro agregaci.
 - `HAVING` : Filtruje skupiny po agregaci.
 - Příklad: `SELECT fakulta, COUNT(*) FROM studenti GROUP BY fakulta HAVING COUNT(*) > 100;`
6. **Transakce a ACID vlastnosti:**
 - Transakce: Logická jednotka práce (vše `COMMIT` nebo nic `ROLLBACK`).
 - ACID: **A**tomicity (vše/nic), **C**onsistency (z konzistentního do konzistentního), **I**solation (nezávislost transakcí), **D**urability (trvalost)

změn).

7. Indexy, Pohledy, Triggery:

- Indexy: Zrychlují čtení dat.
- Pohledy: Virtuální tabulky pro zjednodušení a bezpečnost.
- Triggery: Automaticky spouštěné procedury při událostech pro vynucení pravidel/audit.

Zde je zkrácená verze (tahák) k otázce "Spouště a uložené procedury":

31. Spouště a uložené procedury

Klíčové pojmy

- **Uložená procedura (Stored Procedure):** Pojmenovaný blok SQL kódu, uložený a zkompilovaný v databázi, explicitně volaný.
- **Uložená funkce (Stored Function):** Jako procedura, ale musí vracet skalární hodnotu a lze ji použít v SQL výrazech.
- **Spoušť (Trigger):** Speciální typ uložené procedury, automaticky aktivovaná DML událostí (`INSERT` , `UPDATE` , `DELETE`) na tabulce.
- **Audit:** Záznam změn dat, často realizovaný pomocí spouští.
- `BEFORE` / `AFTER` : Časování spouště (kód se provede před/po DML operaci).
- `IN` / `OUT` / `INOUT` : Typy parametrů procedur (vstupní / výstupní / kombinovaný).
- `OLD` / `NEW` : Pseudotabulární reference ve spouštích pro přístup k původním / novým hodnotám řádku.

A. Uložené procedury (Stored Procedures)

- **Definice:** Pojmenovaný, zkompilovaný SQL kód uložený v DB. Obsahuje deklarace, podmínky (`IF`), cykly (`WHILE`).
- **Volání:** Explicitně z aplikace nebo konzole pomocí `CALL`
`nazev_procedury(parametry);` .
- **Parametry:**
 - `IN` : Vstupní, výchozí. Procedura hodnotu přijímá.
 - `OUT` : Výstupní. Procedura do něj zapíše výsledek pro volajícího.
 - `INOUT` : Vstupní i výstupní. Procedura hodnotu přijímá, modifikuje a vrací upravenou.
- **Návratová hodnota:** Procedura nemusí vracet nic, nebo vrací celou výslednou množinu řádků (`SELECT`). Uložené funkce `musí` vracet jednu skalární hodnotu.
- **Výhody:**
 - **Snížení síťového provozu:** Jediný `CALL` místo mnoha dotazů.

- **Vyšší výkon:** Kompilace a optimalizace DB při uložení.
- **Zvýšení bezpečnosti:** Omezení přímých práv k tabulkám, povolení jen volání procedur.
- **Centralizace logiky:** Konzistentní obchodní pravidla napříč aplikacemi.
- **Modularita a znovupoužitelnost:** Rozdělení komplexních operací.
- **Příklad (MySQL):**

```
sql CREATE PROCEDURE GetStudentInfo (IN student_id INT, OUT student_name VARCHAR(100)) BEGIN SELECT jmeno INTO student_name FROM student WHERE id = student_id; END; -- Volání: CALL GetStudentInfo(1, @name); SELECT @name;
```

B. Spouště (Triggery)

- **Definice:** Speciální procedura, nelze volat ručně. Navázána na tabulku, automaticky se aktivuje na DML událost (`INSERT` , `UPDATE` , `DELETE`).
- **Specifikace:**
 - **Událost:** `INSERT` , `UPDATE` , `DELETE` .
 - **Časování:**
 - `BEFORE` : Před DML. Pro validaci, úpravu dat. Může zabránit operaci.
 - `AFTER` : Po DML. Pro logování, audit, kaskádové aktualizace.
 - **Úroveň:**
 - `FOR EACH ROW` : Spustí se pro každý ovlivněný řádek.
 - `FOR EACH STATEMENT` : Spustí se jednou pro celý příkaz (ne vždy podporováno, např. MySQL jen řádkové).
- **Speciální proměnné:**
 - `OLD` : Hodnoty řádku *před* úpravou (v `UPDATE` , `DELETE`).
 - `NEW` : Hodnoty řádku, které se *budou zapisovat* (v `INSERT` , `UPDATE`).
- **Typické použití:**
 - Vynucování komplexních integritních omezení.
 - Auditování a logování změn dat.
 - Generování odvozených hodnot.
 - Kaskádové operace.
- **Příklad (MySQL):**

```
sql CREATE TRIGGER audit_student_update AFTER UPDATE ON student FOR EACH ROW BEGIN INSERT INTO audit_log (entity, action, old_value, new_value) VALUES ('student', 'UPDATE', OLD.jmeno, NEW.jmeno); END;
```

C. Porovnání a Kdy použít

Vlastnost	Uložená procedura (Stored Procedure)	Spoušť (Trigger)
Aktivace	Explicitní volání (<code>CALL</code>)	Automaticky na DML
Účel	Komplexní logika, transakce, data	Integrita, audit, log
Závislost	Nezávislá na tabulce	Pevně navázaná na
Vstup/Výstup	<code>IN</code> , <code>OUT</code> , <code>INOUT</code> parametry	<code>OLD</code> , <code>NEW</code> pseudo
Kontrola toku	Plná kontrola	Spouští se automa

- **Kdy použít proceduru:** Složitá obchodní logika, transakce, vrácení dat, zvýšení bezpečnosti/výkonu, volání na vyžádání.
- **Kdy použít spoušť:** Automatické vynucování komplexních integritních omezení, auditování, logování, kaskádové akce, logika, která se má provést *vždy* při DML operaci.

Nejčastější chyby u zkoušky

- **Nadměrná aplikační logika v triggerech:** Triggery mají být jednoduché, pro integritu. Složitá logika patří do procedur/aplikace.
- **Neřešení transakčního kontextu:** Chyba v triggeru může způsobit rollback celé transakce.
- **Těžko dohledatelné automatické změny dat:** Triggery mění data "na pozadí", nutná dobrá dokumentace.
- **Ignorování výkonnostních dopadů:** Složité triggery zpomalují DML operace.

Typické zkouškové otázky

1. Co je trigger a kdy se spouští?

- Speciální kód, automaticky aktivovaný DML událostí (`INSERT` , `UPDATE` , `DELETE`) na tabulce. Spouští se `BEFORE` nebo `AFTER` , `FOR EACH ROW`

nebo `FOR EACH STATEMENT` .

2. Jak se liší trigger a stored procedure?

- **Procedura:** Explicitní `CALL` , komplexní logika, transakce, `IN/OUT/INOUT` parametry, vrací data, nezávislá na tabulce.
- **Trigger:** Automaticky na DML událost, integrita, audit, `OLD/NEW` pseudoproměnné, pevně na tabulku.

3. Jaké jsou výhody a nevýhody databázové logiky (procedur a triggerů)?

- **Výhody:** Vyšší výkon, bezpečnost, centralizace logiky, integrita dat, modularita.
- **Nevýhody:** Závislost na DB, obtížnější ladění/testování, skryté chování, zátěž DB.

32. Pohledy, přístupová práva.

Transakce, princip, vlastnosti transakcí.

A. Pohledy (Views)

Pohled (View) je **virtuální tabulka**, která nemá fyzicky uložená data. Je definována uloženým SQL dotazem `SELECT`. Při dotazu na pohled databáze vykoná definovaný dotaz nad podkladovými tabulkami.

Vytvoření a odstranění pohledu

- **Vytvoření:** `sql CREATE VIEW v_student_info AS SELECT s.id, s.jmeno, s.prijmeni, k.nazev AS obor FROM student s JOIN katedra k ON s.katedra_id = k.id WHERE s.aktivni = TRUE;`
- **Odstranění:** `DROP VIEW v_student_info;`

Výhody a využití pohledů

- **Zjednodušení komplexních dotazů:** Zapouzdřuje složité `JOIN` y a agregace.
- **Zvýšení bezpečnosti (Abstrakce dat):** Skrývá citlivá data, zpřístupňuje jen relevantní podmnožinu.
- **Logická nezávislost dat:** Změny v podkladových tabulkách nemusí ovlivnit klientské aplikace (stačí upravit definici pohledu).
- **Updatovatelné pohledy:** Některé jednoduché pohledy (z jedné tabulky, bez agregací) umožňují `INSERT`, `UPDATE`, `DELETE` do podkladové tabulky. Většina je jen pro čtení.

Materiálové (Materializované) pohledy

- Mají data **reálně zkopírovaná a uložená na disku**.
- Používají se pro zrychlení analytických dotazů (datové sklady).
- Nevýhoda: Musí se periodicky aktualizovat (`REFRESH MATERIALIZED VIEW`), data nemusí být real-time aktuální.

B. Přístupová práva

Databázový systém řídí přístup k datům pomocí **DCL (Data Control Language)** příkazů.

Uživatelé a role

- **Uživatel (User):** Konkrétní entita připojující se k databázi.
- **Role (Role):** Skupina oprávnění přidělovaná uživatelům. Zjednodušuje správu práv (RBAC - Role-Based Access Control).

Příkazy DCL

- **GRANT (Udělení práv):**
 - Umožňuje uživateli/roli provádět operace nad objektem.
 - **Příklad:** `GRANT SELECT, INSERT ON student TO 'sekretariat';`
 - Běžná oprávnění: `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `EXECUTE`.
- **REVOKE (Odebrání práv):**
 - Odstraní dříve udělená práva.
 - **Příklad:** `REVOKE INSERT ON student FROM 'sekretariat';`

C. Transakce a jejich vlastnosti (ACID)

Transakce je **nedělitelná posloupnost jednoho nebo více SQL příkazů (DML)**, tvořící jednu logickou jednotku práce. Garantuje, že se provede buď vše, nebo nic.

Řízení transakcí v SQL

- **START TRANSACTION / BEGIN :** Označuje začátek transakce. Změny jsou dočasné.
- **COMMIT :** Úspěšné dokončení transakce. Změny se trvale zapíší na disk a stanou se viditelnými pro ostatní.
- **ROLLBACK :** Zrušení transakce. Všechny dočasné změny se stornují, databáze se vrátí do stavu před transakcí.
- **SAVEPOINT :** Definuje bod v transakci, ke kterému se lze vrátit pomocí `ROLLBACK TO SAVEPOINT`.

Vlastnosti transakcí: Koncept ACID

Každý transakční databázový systém musí splňovat vlastnosti **ACID**:

- **Atomarita (Atomicity - Nedělitelnost):** Transakce je atomický krok. Buď se provedou všechny operace (`COMMIT`), nebo žádná (`ROLLBACK`).
- **Konzistence (Consistency - Soudržnost):** Transakce převede databázi z jednoho validního stavu do jiného validního stavu. Nesmí porušit integritní omezení.
- **Izolovanost (Isolation):** Souběžně běžící transakce se nesmí navzájem ovlivňovat. Navenek se chovají, jako by běžely sekvenčně.
 - **Anomálie:** Dirty Read, Non-repeatable Read, Phantom Read.
 - Zajišťuje se zamykáním (locking) nebo MVCC (Multi-Version Concurrency Control).
- **Trvalost (Durability - Stálost):** Jakmile je transakce potvrzena (`COMMIT`), její změny jsou trvale zapsány na disk a přežijí i výpadek systému (pomocí transakčního logu - WAL).

Typické zkouškové otázky

1. Co je view a k čemu slouží?

- **Odpověď:** Pohled (View) je virtuální tabulka definovaná SQL dotazem. Slouží ke zjednodušení komplexních dotazů, zvýšení bezpečnosti (skrytí citlivých dat) a zajištění logické nezávislosti dat.

2. Vysvětlete ACID.

- **Odpověď:** ACID je akronym pro 4 vlastnosti transakcí:
 - **Atomarita:** Všechny operace transakce se provedou, nebo žádná.
 - **Konzistence:** Transakce převádí DB z validního stavu do validního, bez porušení integritních omezení.
 - **Izolovanost:** Souběžné transakce se navzájem neovlivňují, každá se jeví jako jediná běžící.
 - **Trvalost:** Potvrzené změny jsou trvale uloženy a přežijí selhání systému.

3. Jak funguje COMMIT a ROLLBACK?

- **Odpověď:**

- **COMMIT** : Trvale uloží všechny změny provedené v aktuální transakci do databáze. Změny se stanou viditelnými pro ostatní uživatele a jsou garantovány jako trvalé.
- **ROLLBACK** : Zruší všechny změny provedené v aktuální transakci. Databáze se vrátí do stavu před zahájením transakce. Používá se při chybě nebo nekonzistenci.

33. Indexování a optimalizace dotazů

Klíčové pojmy

- **Index:** Pomocná datová struktura pro zrychlení vyhledávání.
- **Full Table Scan:** Prohledávání celé tabulky.
- **B-strom (B+ Tree):** Vyvážená stromová struktura, nejčastější typ indexu.
- **Hašovací index (Hash Index):** Index založený na hašovací funkci, rychlý pro přesné shody.
- **Clustered Index:** Fyzicky určuje pořadí řádků na disku, max. jeden.
- **Non-Clustered Index:** Uložen odděleně, odkazuje na řádky.
- **Composite Index:** Index nad více sloupci.
- **Query Optimizer:** Komponenta DB, která vybírá nejefektivnější prováděcí plán.
- **Execution Plan:** Sekvence operací pro vykonání dotazu.
- **EXPLAIN:** SQL příkaz pro zobrazení prováděcího plánu.
- **Covering Index:** Index obsahuje všechny sloupce pro dotaz, není nutné sahát do tabulky.

A. Indexy: Účel a cena

Index zrychluje `SELECT` (`WHERE`), `ORDER BY`, `JOIN`.

- **Výhody:** Dramatické zrychlení čtení.
- **Nevýhody:** Zpomaluje `INSERT`, `UPDATE`, `DELETE` (nutná aktualizace indexů); zabírá místo na disku/RAM.
- **Full Table Scan:** Pomalé prohledání celé tabulky, pokud není index použit.

B. Vnitřní struktury indexů

1. B-Strom (B+ Tree):

- Vyvážený strom. Listové uzly obsahují klíče a ukazatele na data, propojeny pro efektivní rozsahové dotazy.
- Složitost: $O(\log n)$.

- Vhodný pro: Přesné shody (`=`), rozsahové dotazy (`<` , `>` , `BETWEEN`), řazení (`ORDER BY`).

2. Hašovací index (Hash Index):

- Využívá hašovací funkci pro mapování hodnot na adresy (buckety).
- Složitost: $O(1)$ (ideálně).
- Omezení: Pouze pro přesnou shodu (`=` , `IN`). Nepoužitelný pro rozsahové dotazy nebo řazení.

C. Typy indexů

- **Clustered Index (Primární index):** Fyzicky určuje pořadí řádků na disku. Listy obsahují přímo data řádků. Max. jeden na tabulku (často nad primárním klíčem).
- **Non-Clustered Index (Sekundární index):** Samostatná datová struktura, uložená odděleně. Obsahuje klíče a ukazatele na řádky (PK nebo fyzická adresa). Více na tabulku.
- **Composite Index (Složený index):** Index vytvořený nad více sloupci (např. (`prijmeni` , `jmeno`)). Pořadí sloupců je klíčové pro využití (prefix matching).

D. Optimalizace dotazů a prováděcí plán

- **Query Optimizer:** Analyzuje dotaz, statistiky o datech, generuje a vybírá nejefektivnější **Execution Plan**.
- **EXPLAIN:** Zobrazí prováděcí plán. Klíčové sloupce:
 - `type` : Způsob prohledávání (např. `ALL` - Full Table Scan, `range` - indexový rozsah, `ref` / `eq_ref` - PK/FK, `const`).
 - `key` : Název použitého indexu.
 - `rows` : Odhadovaný počet prohledaných řádků.
- **Pravidla pro optimalizaci:**
 - Indexujte cizí klíče (`JOIN`).
 - Vyhněte se `SELECT *` (umožňuje využití **Covering Indexu**).
 - `LIKE 'prefix%'` může použít index, `LIKE '%suffix%'` ne.
 - Nepoužívejte funkce nad indexovanými sloupci v `WHERE` (např. `WHERE YEAR(datum) = 2000` -> `WHERE datum BETWEEN '2000-01-01' AND '2000-12-31'`).

Typické zkouškové otázky

1. Index: definice, výhody, nevýhody.

- **Def:** Pomocná DS pro zrychlení vyhledávání.
- **Výhody:** Zrychluje čtení (`SELECT` , `ORDER BY` , `JOIN`).
- **Nevýhody:** Zpomaluje zápis (`INSERT` , `UPDATE` , `DELETE`), zabírá místo.

2. B-strom vs. hašovací index: rozdíly, vhodnost.

- **B-strom:** Vyvážený strom, $O(\log n)$. Pro `=` , rozsahy, řazení.
- **Hašovací:** Hašovací funkce, $O(1)$. Jen pro přesnou shodu (`=` , `IN`).
Nevhodný pro rozsahy/řazení.

3. Clustered vs. non-clustered index. Počet clustered indexů.

- **Clustered:** Fyzicky řadí řádky, listy = data. Max. 1/tabulka.
- **Non-Clustered:** Samostatná DS, odkazuje na řádky. Více/tabulka.

4. Optimalizátor: výběr plánu, analýza. Pravidla optimalizace.

- **Výběr:** Optimalizátor analyzuje dotaz/statistiky, vybírá nejefektivnější plán.
- **Analýza:** `EXPLAIN` (sleduje `type` , `key` , `rows`).
- **Pravidla:** Indexujte FK; vyhněte se `SELECT *` ; žádné funkce nad index. sloupci v `WHERE` .